

Московский Авиационный Институт
(Национальный Исследовательский Университет)
Институт №8 “Компьютерные науки и прикладная математика”
Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №1 по курсу
«Операционные системы»

Группа: М8О-214БВ-25

Студент: Мозговая Ж.Е.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 06.10.26

Москва, 2026

Постановка задачи

Вариант 12.

Составить и отладить программу на языке Си, осуществляющую работу с процессами и взаимодействие между ними. Родительский процесс создаёт два дочерних процесса. Перенаправление стандартных потоков ввода-вывода — согласно схеме (parent → pipe1 → child1 → child2 → pipe2 → parent). Child1 переводит строки в верхний регистр. Child2 убирает все задвоенные пробелы. Родительский и дочерний процессы представлены разными программами. Необходимо обрабатывать системные ошибки.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `pid_t fork(void)`; — создаёт дочерний процесс, полную копию родителя. Копирует таблицу файловых дескрипторов. Возвращает 0 в ребёнке, PID ребёнка в родителе, -1 при ошибке.
- `int pipe(int fd[2])`; — создаёт однонаправленный канал. `fd[0]` — read-конец, `fd[1]` — write-конец. Возвращает 0 при успехе, -1 при ошибке.
- `int dup2(int oldfd, int newfd)`; — дублирует дескриптор `oldfd` в `newfd`, при необходимости закрывая `newfd`. Используется для перенаправления `stdin/stdout`.
- `int execl(const char *path, const char *arg, ..., NULL)`; — заменяет текущий процесс новой программой. Таблица дескрипторов сохраняется.
- `ssize_t read(int fd, void *buf, size_t count)`; — читает до `count` байт из `fd` в `buf`. Блокируется при отсутствии данных. Возвращает 0 при EOF.
- `ssize_t write(int fd, const void *buf, size_t count)`; — пишет до `count` байт из `buf` в `fd`. Блокируется при заполненном буфере `pipe`.
- `int close(int fd)`; — закрывает дескриптор. Необходим для корректного EOF: read-конец получает EOF только когда все write-концы закрыты.
- `pid_t waitpid(pid_t pid, int *status, int options)`; — ожидает завершения дочернего процесса. Устраняет зомби-процессы.
- `void perror(const char *s)`; — печатает сообщение об ошибке с описанием из `errno`.
- `void exit(int status)`; / `void _exit(int status)`; — завершают процесс. `_exit` используется в ребёнке после неудачного `exes`, чтобы избежать дублирования буферов `stdio`.

В рамках лабораторной работы была разработана программа, реализующая обмен данными между процессами через неименованные каналы (`pipe`) в среде Linux. Программа состоит из трёх отдельных исполняемых файлов: `parent`, `child1` и `child2`. Разделение на разные программы соответствует требованию задания о том, что родительский и дочерние процессы должны быть представлены разными программами.

Родительский процесс (parent) выполняет роль управляющего звена:

1. Создает три канала (pipe, pipe, pipe), получая шесть дескрипторов.
2. Через fork создает два дочерних процесса и в каждом из них с помощью dup2 подменяет стандартные потоки ввода-вывода на нужные концы каналов, после чего вызовом execl запускает соответствующую дочернюю программу.
3. Закрывает все ненужные ему концы каналов, оставляя только write-конец первого pipe (для передачи строк в child1) и read-конец третьего pipe (для приема результата от child2).
4. В цикле читает строки от пользователя через getline, передает их в первый pipe, читает результат из третьего pipe и выводит его в стандартный поток вывода.
5. Пустая строка служит признаком завершения. После её ввода родитель закрывает write-конец первого pipe, что приводит к каскадному завершению всей цепочки дочерних процессов.
6. Ожидает завершения обоих детей через waitpid, устраняя зомби-процессы.

Первый дочерний процесс (child1) читает данные из своего стандартного ввода (который через dup2 подключён к pipe1), переводит каждый символ в верхний регистр с помощью toupper и записывает результат в стандартный вывод (подключён к pipe2). Процесс работает с блоками байт, а не со строками, поэтому использует буфер фиксированного размера и читает до тех пор, пока read не вернёт 0 (EOF).

Второй дочерний процесс (child2) читает данные из pipe2, убирает задвоенные пробелы (подряд идущие символы ' ' схлопываются в один) и записывает результат в pipe3. Для отслеживания пробелов на стыке блоков используется флаг prev_space, объявленный вне цикла чтения. Поскольку длина результата может быть меньше длины входных данных, выход формируется в отдельном буфере.

Обмен данными между процессами осуществляется исключительно через pipe-каналы. Каждый pipe однонаправленный и имеет два конца: read-конец и write-конец. Синхронизация процессов обеспечивается ядром: если данные отсутствуют, read блокирует процесс; если буфер канала заполнен, write блокирует процесс. При закрытии всех write-концов канала читатель получает признак конца данных (EOF, read возвращает 0).

Обработка ошибок реализована для всех системных вызовов: pipe, fork, dup2, read, write, close, execl, waitpid. При ошибке любого из них выводится сообщение через perror с описанием из errno, после чего программа завершается с ненулевым кодом.

Каскадное завершение цепочки процессов работает следующим образом: после ввода пустой строки родитель закрывает write-конец pipe1, в результате чего child1, прочитав оставшиеся данные, получает EOF и завершается. При завершении child1 ядро автоматически закрывает все его дескрипторы, включая write-конец pipe2. Child2 получает EOF на pipe2, завершается, его выходной pipe3 закрывается — родитель может закончить работу.

Код программы

parent.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>
#include <sys/wait.h>

static void die(const char *msg) { perror(msg); exit(EXIT_FAILURE); }

static void close_all(int *p1, int *p2, int *p3) {
    int *arr[3] = { p1, p2, p3 };
    for (int a = 0; a < 3; a++)
        for (int i = 0; i < 2; i++)
            if (arr[a][i] > 2) close(arr[a][i]);
}

static pid_t spawn(const char *path, int in_fd, int out_fd,
                  int *p1, int *p2, int *p3) {
    pid_t pid = fork();
    if (pid < 0) die("fork");

    if (pid == 0) {
        if (dup2(in_fd,  STDIN_FILENO) < 0) die("dup2 in");
        if (dup2(out_fd, STDOUT_FILENO) < 0) die("dup2 out");
        close_all(p1, p2, p3);
        execl(path, path, NULL);
        perror("execl"); _exit(EXIT_FAILURE);
    }
    return pid;
}

int main(void) {
    int p1[2], p2[2], p3[2];
    if (pipe(p1) < 0) die("pipe1");
    if (pipe(p2) < 0) die("pipe2");
    if (pipe(p3) < 0) die("pipe3");

    pid_t c1 = spawn("./child1", p1[0], p2[1], p1, p2, p3);
    pid_t c2 = spawn("./child2", p2[0], p3[1], p1, p2, p3);

    close(p1[0]); close(p2[0]); close(p2[1]); close(p3[1]);
    int wfd = p1[1], rfd = p3[0];

    printf("Enter lines (empty line to finish):\n");

    char *line = NULL;
    size_t cap = 0;
    ssize_t len;

    while ((len = getline(&line, &cap, stdin)) > 0) {
        if (len == 1 && line[0] == '\n') break;
        if (write(wfd, line, len) != len) die("write to pipe1");

        char out[4096];
```

```

        ssize_t got = 0;
        while (got < (ssize_t)sizeof(out) - 1) {
            ssize_t r = read(rfd, out + got, 1);
            if (r < 0) die("read from pipe3");
            if (r == 0) break;
            if (out[got++] == '\n') break;
        }
        if (got > 0) {
            out[got] = '\0';
            printf("Result: %s", out);
        }
    }
    free(line);
    close(wfd);

    waitpid(c1, NULL, 0);
    waitpid(c2, NULL, 0);
    return 0;
}

```

child1.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <ctype.h>
#include <errno.h>

int main(void) {
    char buf[4096];
    ssize_t n;

    while ((n = read(STDIN_FILENO, buf, sizeof(buf))) > 0) {
        for (ssize_t i = 0; i < n; i++)
            buf[i] = (char)toupper((unsigned char)buf[i]);

        if (write(STDOUT_FILENO, buf, n) != n) {
            perror("child1: write");
            return EXIT_FAILURE;
        }
    }
    if (n < 0) { perror("child1: read"); return EXIT_FAILURE; }
    return 0;
}

```

child2.c

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <errno.h>

int main(void) {
    char in[4096];
    char out[4096];
    ssize_t n;
    int prev_space = 0;

```

```

while ((n = read(STDIN_FILENO, in, sizeof(in))) > 0) {
    ssize_t k = 0;
    for (ssize_t i = 0; i < n; i++) {
        if (in[i] == ' ') {
            if (prev_space) continue;
            prev_space = 1;
        } else {
            prev_space = 0;
        }
        out[k++] = in[i];
    }
    if (write(STDOUT_FILENO, out, k) != k) {
        perror("child2: write");
        return EXIT_FAILURE;
    }
}
if (n < 0) { perror("child2: read"); return EXIT_FAILURE; }
return 0;
}

```

Протокол работы программы

```

gcc -Wall -Wextra -O2 -std=gnu11 -o parent parent.c
gcc -Wall -Wextra -O2 -std=gnu11 -o child1 child1.c
gcc -Wall -Wextra -O2 -std=gnu11 -o child2 child2.c
mozgovay@DESKTOP-HLE2R5I:~/lab1$ ./parent
Enter lines (empty line to finish):
bvjrvb   rvr
Result: BVJRVB RVR

mozgovay@DESKTOP-HLE2R5I:~/lab1$ ./parent
Enter lines (empty line to finish):
nrevkr           rjngel
Result: NREVKR RJNGEL
vejnv'
Result: VEJNV'
bb
Result: BB
bbbbbek   we   e e e fbe evv ev           e e e
Result: BBBBBEK WE E E E FBE EVV EV E E E

mozgovay@DESKTOP-HLE2R5I:~/lab1$ ./parent
Enter lines (empty line to finish):
BBBB b BB
Result: BBBB B BB

```

Вывод

В ходе лабораторной работы были изучены и практически применены системные вызовы для управления процессами и организации межпроцессного взаимодействия: `fork`, `pipe`, `dup2`, `execl`, `read`, `write`, `close`, `waitpid`. Разработана программа, реализующая конвейер из трёх процессов, обменивающихся данными через `pipe`-каналы: `child1` переводит строки в верхний регистр, `child2` удаляет задвоенные пробелы, родитель передаёт данные между ними и выводит результат пользователю. Все системные вызовы сопровождаются обработкой ошибок через `perror`.

Основные сложности при выполнении работы были связаны с корректным закрытием лишних концов pipe-каналов после `fork` (иначе не наступает EOF), а также с изменением длины строки в процессе обработки — из-за схлопывания пробелов родитель не может читать фиксированное число байт и вынужден ориентироваться на символ `\n`. Для отладки многоступенчатого конвейера оказался незаменим инструмент `strace`, позволивший быстро найти причину «зависания» программы на системном вызове `read`.